



Engs 28

# Embedded Systems

Day 4

# Agenda

Announcements

Code review: Set up your Nucleo to show the LED toggle code.

State machine design pattern

# Announcements

## Labs:

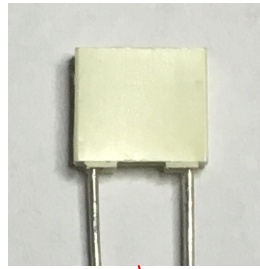
- Start early! Greater chance of getting 1-1 TA time on Tuesday, Wednesday, Thursday.
- Signatures: try your best to get an in-person signature. Video submission should be used in an emergency only.
  - If something is off with your demo and at signature time it is noticed by a TA, you still have the chance to fix it. -> better chance for more points
  - Videos made on apple devices are difficult to view for TAs with Windows computers.
- Follow the submission instructions in the lab.

# Announcements

## Quiz 1:

- Available anytime between noon on Sunday, January 18, and noon on Tuesday, January 20.
  - Going forward, quizzes will be available on Mondays, 12:01 am-11:59 pm.
- The time limit is 60 minutes. You may take the quiz in any uninterrupted one-hour period during the quiz availability.
- Permitted sources: lecture and lab handouts, readings in Williams and White the Nucleo reference manual, datasheet, anything else archived on Canvas.

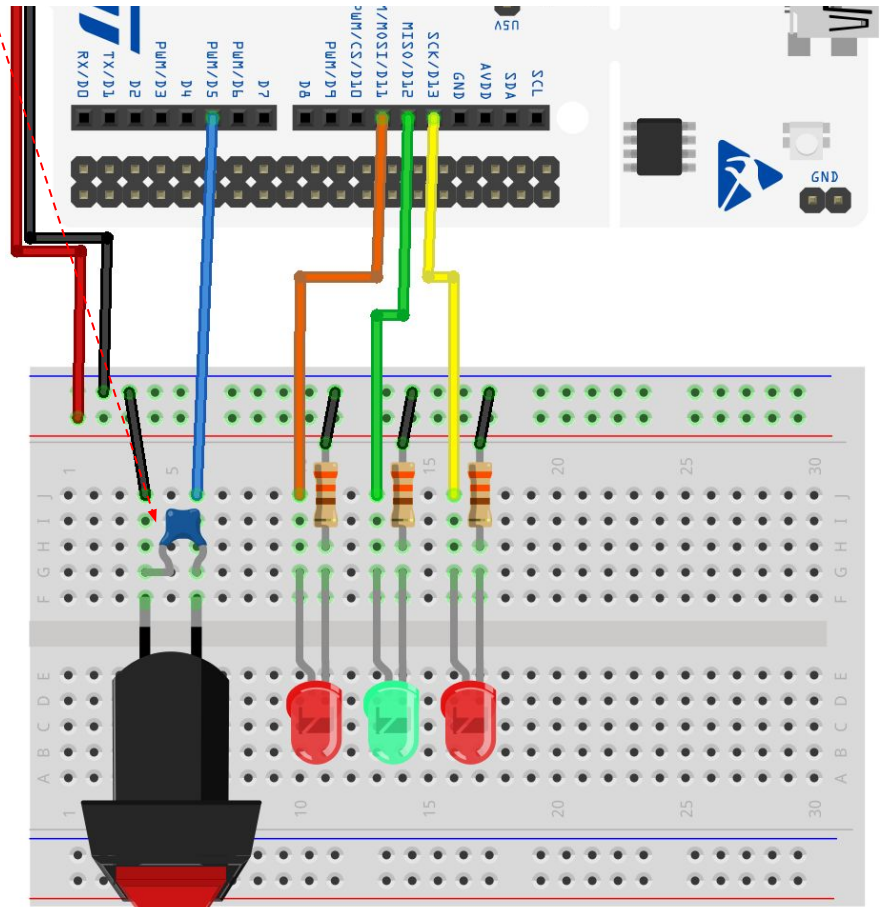
# Homework



Write code (toggleLED.c) so that the on-board LED is toggled on and off via the button:

PRESS     => LED goes ON  
RELEASE => LED stays ON  
PRESS     => LED goes OFF  
RELEASE => LED stays OFF

In order to prevent “glitches” (erratic behavior of the LED caused by button bounce), add a 0.15 $\mu$ F capacitor across the switch.



# Code Review

- Share code within your group.
- Can you read, understand, and explain your partners' codes? (This is called "code review" — good for catching bugs!)

```

#include "ES28.h"                // All the port definitions are here

#define GPIOAEN      (1U<<0)    // Clock access to GPIOA (LED)
#define GPIOBEN      (1U<<1)    // Clock access to GPIOB (button)

#define LED_PIN      (1U<<5)    // LED on PA5      (D13)
#define BUTTON_PIN    (1U<<4)    // Button on PB4 (D5)

int main(void) {
    unsigned buttonAlreadyPressed = 0;
    unsigned buttonPushed;
    RCC->IOPENR |= (GPIOAEN | GPIOBEN);    // Enable clock access to GPIOA,B

    GPIOA->MODER |= (1U<<10);    // Configure PA5 as output pin
    GPIOA->MODER &= ~(1U<<11);

    GPIOB->MODER &= ~(1U<<8);    // Configure PB4 as input pin
    GPIOB->MODER &= ~(1U<<9);    // (not necessary since this is default reset value)

    GPIOB->PUPDR |= (1U<<8);    // enable pull-up for PB4
    GPIOB->PUPDR &= ~(1U<<9);

    while(1) {
        buttonPushed = ((GPIOB->IDR & BUTTON_PIN) == 0); // get state of button pin (low/high)
        if (!buttonAlreadyPressed) {
            if (buttonPushed) {
                GPIOA->ODR ^= LED_PIN;    // toggle LED
                buttonAlreadyPressed = 1;
            }
        } else {
            if (!buttonPushed) {
                buttonAlreadyPressed = 0;
            }
        }
    }
    return 0;    // never reached
}

```

# Control the LED

- toggle -  
possible  
solution 1

# Control the LED - toggle - possible solution 2

```
#include "ES28.h"                // All the port definitions are here

#define GPIOAEN      (1U<<0)    // Clock access to GPIOA (LED)
#define GPIOBEN      (1U<<1)    // Clock access to GPIOB (button)

#define LED_PIN      (1U<<5)    // LED on PA5      (D13)
#define BUTTON_PIN    (1U<<4)    // Button on PB4  (D5)

int main(void) {
    unsigned buttonAlreadyPressed = 0;
    unsigned buttonPushed;
    RCC->IOPENR |= (GPIOAEN | GPIOBEN);    // Enable clock access to GPIOA,B

    GPIOA->MODER |= (1U<<10);    // Configure PA5 as output pin
    GPIOA->MODER &= ~(1U<<11);

    GPIOB->MODER &= ~(1U<<8);    // Configure PB4 as input pin
    GPIOB->MODER &= ~(1U<<9);    // (not necessary since this is default reset value)

    GPIOB->PUPDR |= (1U<<8);    // enable pull-up for PB4
    GPIOB->PUPDR &= ~(1U<<9);

    while(1) {
        buttonPushed = ((GPIOB->IDR & BUTTON_PIN) == 0);
        if (buttonPushed) {
            if (buttonAlreadyPressed==0) {
                GPIOA->ODR ^= LED_PIN;    // toggle LED
                buttonAlreadyPressed = 1;
            }
        } else {
            buttonAlreadyPressed = 0;
        }
    }
    return 0;                    // never reached
}
```



# State machine design pattern

```

#include "ES28.h"                // All the port definitions are here

#define GPIOAEN      (1U<<0)    // Clock access to GPIOA (LED)
#define GPIOBEN      (1U<<1)    // Clock access to GPIOB (button)

#define LED_PIN      (1U<<5)    // LED on PA5      (D13)
#define BUTTON_PIN   (1U<<4)    // Button on PB4 (D5)

int main(void) {
    unsigned buttonAlreadyPressed = 0;
    unsigned buttonPushed;
    RCC->IOPENR |= (GPIOAEN | GPIOBEN); // Enable clock access

    GPIOA->MODER |= (1U<<10);    // Configure PA5 as output pin
    GPIOA->MODER &= ~(1U<<11);

    GPIOB->MODER &= ~(1U<<8);    // Configure PB4 as input pin
    GPIOB->MODER &= ~(1U<<9);    // (not necessary since this is default reset value)

    GPIOB->PUPDR |= (1U<<8);    // enable pull-up for PB4
    GPIOB->PUPDR &= ~(1U<<9);

    while(1) {
        buttonPushed = ((GPIOB->IDR & BUTTON_PIN) == 0);
        if (!buttonAlreadyPressed) {
            if (buttonPushed) {
                GPIOA->ODR ^= LED_PIN;
                buttonAlreadyPressed = 1;
            }
        } else {
            if (!buttonPushed) {
                buttonAlreadyPressed = 0;
            }
        }
    }
    return 0;                    // never reached
}

```

# Look again at toggling the LED with pushbutton

**STATE:** buttonAlreadyPressed  
 0: switch unpressed  
 1: switch has been pressed

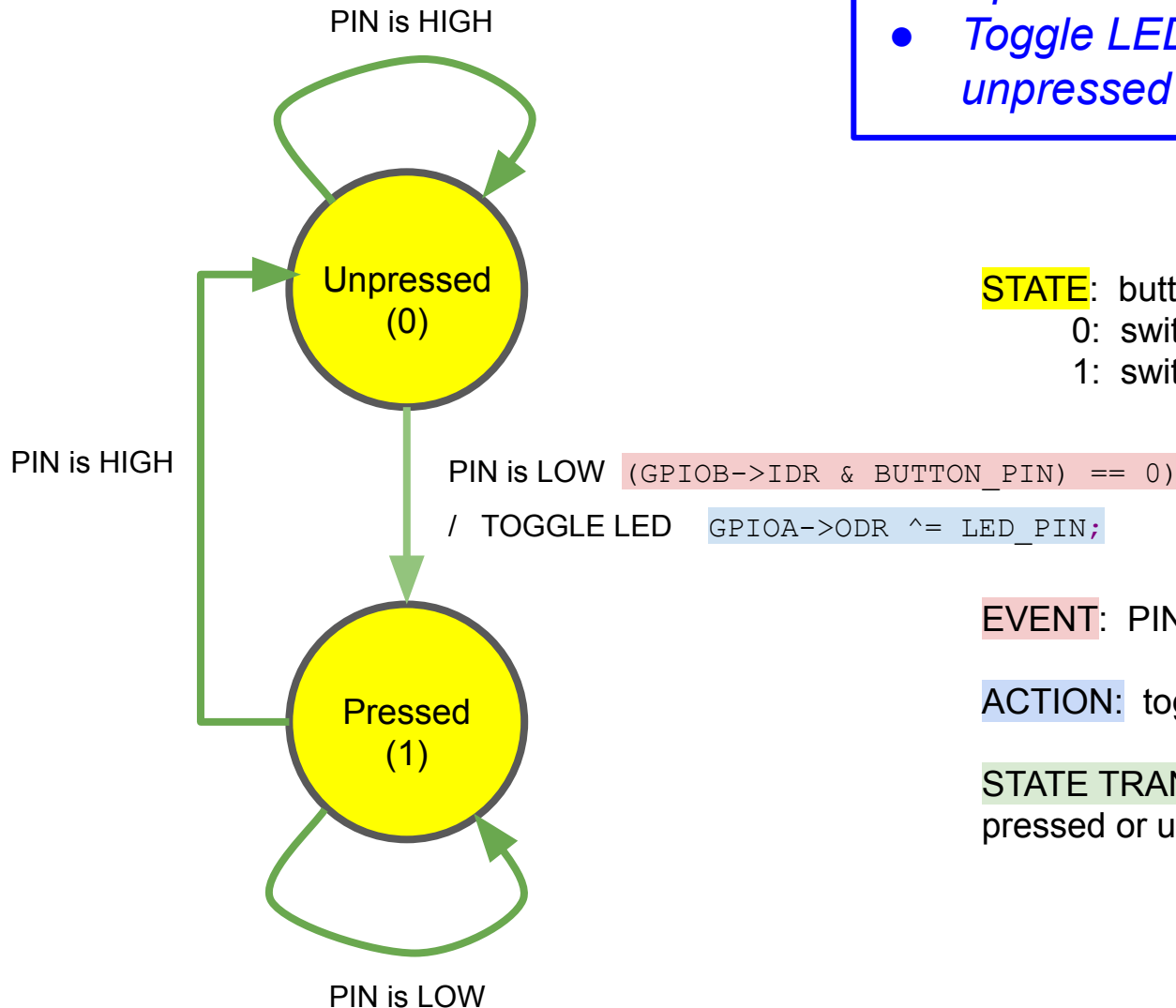
**EVENT:** pin detected low or high

**ACTION:** toggle the LED

**STATE TRANSITION:** mark button as pressed or unpressed

# State diagram

Capture behavior of system graphically



*The button can be pressed or unpressed. But it is only when we observe a transition that we need to do anything:*

- *Update the state*
- *Toggle LED in transition from unpressed to pressed.*

**STATE:** buttonAlreadyPressed  
0: switch unpressed  
1: switch has been pressed

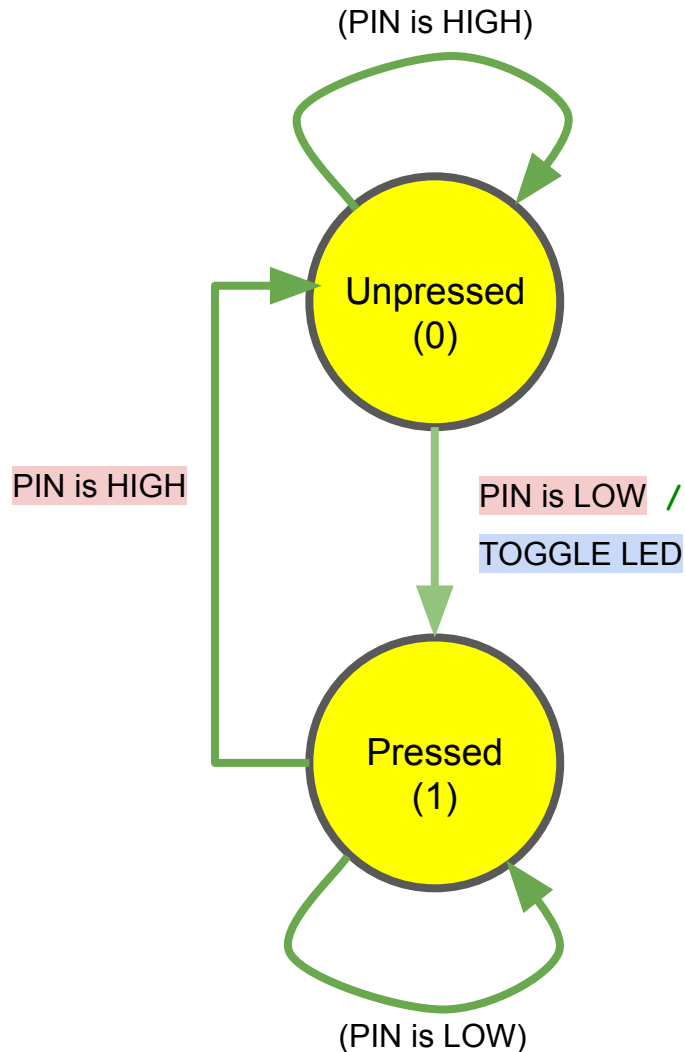
**EVENT:** PIN detected LOW or HIGH

**ACTION:** toggle the LED

**STATE TRANSITION:** mark button as pressed or unpressed

# Translate state diagram to C

Enumerated data type makes it easy to write code that mirrors the state diagram!



```
typedef enum {UNPRESSED, PRESSED} state_t ;
state_t state = UNPRESSED;

...
while(1) {
    buttonPushed = ((GPIOB->IDR & BUTTON_PIN) == 0);
    switch(state) {
        case UNPRESSED:
            if (buttonPushed) {
                GPIOA->ODR ^= LED_PIN;
                state = PRESSED;
            }
            break;
        case PRESSED:
            if (!buttonPushed) {
                state = UNPRESSED;
            }
            break;
    }
}
```

# Lab 2 design challenges

1. Use a pushbutton to pause / resume a counter (begin with blinkyCNT.c, adapt LED-toggling state machine).
2. Reaction time game: Each player has a (debounced) button and a red “WIN” LED. A third (green) LED serves as the start signal.
  1. Initially, the starter LED is ON, or perhaps blinking rapidly.
  2. Players can press either button to start the game.
  3. The starter LED “counts down” with three flashes, one second apart, then glows steadily.
  4. The players press their buttons as soon as they see the starter LED become steady. The player who presses their button first wins, and their LED lights (or blinks rapidly) for a second.
  5. The system returns to step 1.

Use state machine design pattern. Begin with paper design / state diagram.

# Lastly, Don't Forget:

|   |                         |                                                |                                                                                                                                          |                                                                                                                                                                                                                               |
|---|-------------------------|------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 3 | Mon                     |                                                | <a href="#">Quiz 1</a>                                                                                                                   |                                                                                                                                                                                                                               |
|   | <a href="#">Tues</a> ↓  | <a href="#">5. Serial communication (UART)</a> | <a href="#">RQ Day 5</a><br><a href="#">"Practice Debugging" activity</a><br><a href="#">Install CoolTerm</a> ➡<br><a href="#">Lab 2</a> | <a href="#">uart.c</a> ↓ <a href="#">uart.h</a> ↓<br><a href="#">counter.c</a> ↓                                                                                                                                              |
|   | <a href="#">Wed</a> ↓   | 5x. Datasheets                                 |                                                                                                                                          | <a href="#">Datasheet Activity Slides</a> ➡<br><a href="#">humidity sensor</a> ↓ <a href="#">temp sensor</a> ↓<br><a href="#">hall effect sensor</a> ↓ <a href="#">real time clock</a> ↓<br>↓ <a href="#">accelerometer</a> ↓ |
|   | <a href="#">Thurs</a> ↓ | <a href="#">6. Driving Large Loads</a>         | <a href="#">RQ Day 6</a><br><a href="#">Upload Keyboard Counter code</a>                                                                 |                                                                                                                                                                                                                               |